EkoWatch Build Plan

Prepared: 30 October 2026 · Team 1 · Implementation baseline v1

1 Purpose authority and scope

Prevent the naming and integration disagreements encountered on FoodBridge by agreeing database names, Java APIs/controllers, HTTP payloads and frontend types before implementing a slice.

Requirements source: Tesa cohort 6 software engineering capstone project.pdf, page 14 (side-project standard), pages 15–16 (EkoWatch), pages 2–3 (ownership and shared fairness rules), and pages 4–5 (checkpoint). The original Monday checkpoint was 26 October 2026; it is historical, not a new deadline. Confirm the final submission date with the facilitator.

The user refers to the brief as intent.md. A separate file with that name was not located. This document uses the original PDF; it does not claim to have audited an existing intent.md or the current repositories. Before applying it, compare existing schema, controllers and frontend calls, then record intentional migrations and compatibility changes.

This document defines one naming convention and one implementation baseline for the team. Names in the mappings are the selected names to implement. Product obligations come from the brief; implementation policies such as the six hour freshness window are decisions made in this plan, not additional assessment requirements. Existing code has not been audited and team approval is not implied.

EkoWatch provides community reports and collection records for one ward. It does not forecast floods, dispatch crews, replace an agency or certify that a road is safe. FoodBridge remains the main product; guide effort approximately two thirds FoodBridge, one third EkoWatch.

2 Ownership and contract governance

Area	Primary owner	Required handoff
Data and rules	Adedayo	Schema, migrations, rule implementation/tests, seed data and logging; review payload fields with Esther
API and access	Esther	OpenAPI, controllers, validation, authentication, authorisation, pipeline, deployments and API verification
Frontend	Muizz	Routes, typed client, forms, all journey states, keyboard access and responsive screens
Joint decisions	All three	Resolve naming and business semantics before either tier depends on them

Keep the Java 25 modular monolith in its own EkoWatch repository, separate from FoodBridge. This is a project-context choice, not a stack requirement in the brief.

Repository documents:

File	Purpose
intent.md	Approved requirements, transcribed or referenced with PDF page numbers; preserve Required/Should distinctions
build.md	This plan, vocabulary, cross-tier mappings, sequencing and acceptance

File	Purpose
openapi.yaml	Executable HTTP source of truth: paths, operationId, schemas, required fields, nullability, errors and examples
docs/decisions.md	Dated contract changes, rationale and named backend/frontend agreement
docs/adr/001-confirmations-and-freshness.md	First decision record
docs/adr/002-disputed-pickups.md	Second decision record
docs/ai-log.md	Tools, prompts, accepted/rejected work and checks
README.md	Live URLs, setup, migration/seed commands, demo logins, fictional-data labels and limitations

For each change: edit OpenAPI and this mapping → backend and frontend reviewers agree → update Java/TypeScript and fixtures → verify → record decision. Do not independently rename type to category, collectionDate to pickupDate, or confirmationCount to votes. Database snake_case is an intentional mapping, not JSON naming.

3 Required scope across all three sectors

E01
Required capability: Three roles: resident, operator, authority

Database/data rules: Users and role constraints; password hashes

API/controllers: Authenticate; enforce every permission on service calls

Frontend: Role-aware navigation; handle 401/403

Acceptance evidence: Direct wrong-role request refused; test fails with check removed

E02
Required capability: Seeded ward and street selection

Database/data rules: One ward; at least eight linked streets

API/controllers: Ward/street lookup and resident preference

Frontend: Select ward then street; handle empty lookup

Acceptance evidence: Selection survives refresh; wrong ward/street pair refused

E03
Required capability: Collection days per street

Database/data rules: Dated schedule occurrences and collection records

API/controllers: Return derived state; operator records completion

Frontend: Schedule and assigned-street screen

Acceptance evidence: Past scheduled date without completion displays missed

E04
Required capability: Four report types with note, street, optional photo/location

Database/data rules: Restricted enum; reporter private; optional evidence fields

API/controllers: Validate type, street and evidence; actor from login

Frontend: Validated report form, denied-location fallback

Acceptance evidence: All four types can be created; no location/photo still works

E05

Required capability: One confirmation per resident/report; higher count ranks higher

Database/data rules: Unique (report_id,resident_id)

API/controllers: Atomic confirmation; no lost concurrent counts

Frontend: Confirm action, count, confirmed state and ranking

Acceptance evidence: Two people count twice; same person counts once

E06

Required capability: Flood current/expired based on latest evidence

Database/data rules: Persist evidence times; derive freshness

API/controllers: Compute at read time using server clock and fixed window

Frontend: Current/expired badge, last evidence age

Acceptance evidence: Expiry works after outage, without scheduler

E07

Required capability: Flood map and usable list alternative

Database/data rules: Optional public street coordinates

API/controllers: Current floods and blocked drains

Frontend: List works if map fails or is unavailable

Acceptance evidence: Same relevant reports in list; newest report age shown

E08

Required capability: Ward authority dashboard

Database/data rules: Derive from records; no hard-coded totals

API/controllers: Authority-only counts and operator performance

Frontend: Ward totals, operator breakdown, evidence links

Acceptance evidence: Resident/operator cannot access performance data

E09

Required capability: Anonymous report display and photo privacy

Database/data rules: Private reporter FK; no GPS metadata in saved images

API/controllers: Allowlisted public DTO; sanitise before storing photo

Frontend: No reporter name; explain community evidence

Acceptance evidence: Scan responses and stored image metadata

E10

Required capability: Once per street/day, assigned operators only

Database/data rules: Unique (street_id,collection_date); ward assignment

API/controllers: Check assignment and schedule inside transaction

Frontend: Only assigned streets; duplicate completion feedback

Acceptance evidence: Concurrent writes produce one record; foreign ward refused

E11

Required capability: Disputed operator/resident claims

Database/data rules: Preserve both completion and missed-pickup report

API/controllers: Derive dispute, do not erase either record

Frontend: Clear “Disputed” label with both evidence times

Acceptance evidence: Seed and demonstrate one disputed pickup

E12
Required capability: Required delivery and usability

Database/data rules: Reproducible migration and awkward seeds

API/controllers: Two deployed tiers via gated GitHub Actions

Frontend: Five states, keyboard/focus, phone layout

Acceptance evidence: Clean-browser deployed journey and CI evidence

Optional Shoulds: missed-pickup in-app alerts, rain-ready reminder, overlap hotspot view and pin-drop setup. Later: more wards, commuter route alerts, official forecasts, SMS/USSD and payments for extra pickups. Do not add them to MVP acceptance.

4 Canonical vocabulary and representation

Ward identity
Database: ward_id

Java: wardId

JSON and TypeScript: wardId: string

Street identity
Database: street_id

Java: streetId

JSON and TypeScript: streetId: string

Report identity
Database: report_id

Java: reportId

JSON and TypeScript: reportId: string

Report kind
Database: report_type

Java: ReportType reportType

JSON and TypeScript: reportType: ReportType

Reporter (private)
Database: reporter_id

Java: reporterId internally

JSON and TypeScript: Never in public report DTO

Confirmation total
Database: Derived COUNT

Java: confirmationCount

JSON and TypeScript: confirmationCount: number

Viewer confirmation
Database: Derived EXISTS

Java: confirmedByMe

JSON and TypeScript: confirmedByMe: boolean

Collection calendar date
Database: collection_date

Java: LocalDate collectionDate

JSON and TypeScript: collectionDate: string (YYYY-MM-DD)

Collection result
Database: Derived

Java: CollectionStatus collectionStatus

JSON and TypeScript: collectionStatus: CollectionStatus

Flood freshness
Database: Derived

Java: FloodStatus floodStatus

JSON and TypeScript: floodStatus: FloodStatus | null

Latest evidence
Database: Derived maximum evidence time

Java: lastEvidenceAt

JSON and TypeScript: lastEvidenceAt: string

Optional image
Database: photo_id

Java: photoUrl in view

JSON and TypeScript: photoUrl: string | null

Optional precise location
Database: latitude, longitude private

Java: request location

JSON and TypeScript: { latitude: number; longitude: number } | null on create only

Request tracing
Database: Log field requestId

Java: requestId

JSON and TypeScript: Header X-Request-ID, error requestId

Naming rules: UUID IDs represented as strings; snake_case tables/columns; PascalCase Java and TypeScript types; lowerCamelCase JSON and methods; kebab-case multiword URL segments; plural resource URLs. Java uses Instant/Postgres timestamptz for instants, sent as ISO 8601 UTC. Collection dates and boundaries use Africa/Lagos, not the browser timezone. Counts are integers. Unknown enum values are invalid; no silent aliases.

Exact shared enums:

```
type Role = 'RESIDENT' | 'OPERATOR' | 'AUTHORITY';
type ReportType = 'MISSED_PICKUP' | 'ILLEGAL_DUMPING' | 'BLOCKED_DRAIN' | 'FLOODED_ROAD';
type FloodStatus = 'CURRENT' | 'EXPIRED';
type CollectionStatus = 'SCHEDULED' | 'DONE' | 'MISSED' | 'DISPUTED';
```

The brief does not require report resolution/closure. Do not introduce an ambiguous report status enum for MVP. Flood freshness and collection outcome are distinct concepts.

5 Database contract

These are the canonical implementation names. Migrate conflicting existing names explicitly. All domain foreign keys are NOT NULL unless explicitly optional. Give constraints stable names so failures map predictably to errors.

Table	Columns and relationships	Protection
users	id, display_name, email, password_hash, role, nullable ward_id, nullable street_id, created_at	Unique normalised email; role check; street belongs to selected ward
wards	id, name	Unique name within this one-ward MVP
streets	id, ward_id, name, nullable public latitude,longitude	FK ward; unique (ward_id,name)
operator_wards	operator_id, ward_id	Composite PK; service verifies OPERATOR role
collection_schedules	id, street_id, operator_id, collection_date, created_at	uq_schedule_street_date(street_id,collection_date) ; FK assignment; role validated in service
collections	id, street_id, collection_date, completed_by, completed_at	uq_collection_street_date(street_id,collection_date); composite FK to scheduled street/date
photos	id, owner_id, storage_key, content_type, size_bytes, created_at	Only server-processed bytes; no original with metadata retained
reports	id, report_type, street_id, reporter_id, note, nullable photo_id, nullable latitude,longitude, nullable collection_date, created_at	Report-type check; location pair/range checks; MISSED_PICKUP requires scheduled collection date; other types require null date
report_confirmations	id, report_id, resident_id, confirmed_at	uq_report_confirmation_resident(report_id,resident_id); FKs

Use schedule occurrences rather than introducing recurrence rules: seed thirty consecutive calendar days of coverage with explicit collection dates for each street's collection pattern. Display unscheduled dates as “No collection scheduled”; they do not become missed. Historical operator_id on the occurrence keeps past performance attached to the operator responsible then.

Suggested indexes: streets(ward_id,name); collection_schedules(street_id,collection_date) via unique constraint; collection_schedules(operator_id,collection_date); reports(street_id,created_at DESC,id); reports(report_type,created_at DESC,id); confirmation unique index. Do not add a reverse confirmation index for MVP. The unique confirmation index already supports report counts and maximum evidence time, although it may still scan many confirmations. Do not promise constant-time aggregate queries. Avoid a standalone reports(latitude) index for MVP: no radius search is required.

Migrations must create the schema and constraints reproducibly on PostgreSQL. Demo seeds are separate from production migrations and rerunnable without duplicate natural keys. Do not edit an already-deployed versioned migration; add a new one. Document recovery for failed deployment migrations.

6 Business decisions to adopt and record

Confirmation and flood freshness

Required: another resident confirms a report; two different residents count twice and repeat confirmation by one person counts once. Reject self-confirmation with 409 SELF_CONFIRMATION_NOT_ALLOWED. Use the logged-in resident identity; never accept a residentId from the browser.

The endpoint is PUT /api/v1/reports/{reportId}/confirmations/me. It is an idempotent evidence submission: insert first confirmation, or refresh that resident's confirmed_at only when the flood was already expired at the start of the transaction. Repeated calls while current are no-ops. Lock the report row before checking freshness and inserting/updating, so concurrent reconfirmations cannot make inconsistent choices. The unique constraint remains essential even with the lock.

This permits an existing confirmer to reconfirm an expired flood without adding a second row or inflating the unique-person count. A plain retry after a successful reconfirmation is a no-op while current. A much later new submission after expiry is fresh evidence; this is an explicitly documented time-sensitive PUT semantic, not unconditional byte-for-byte replay forever. No confirmation removal in MVP.

Freshness configuration: FLOOD_FRESHNESS_HOURS=6 (the brief specifies a set window, not six hours). lastEvidenceAt = max(report.createdAt, max(confirmedAt)); expiresAt = lastEvidenceAt + window; CURRENT iff serverTime < expiresAt, otherwise EXPIRED. Return floodStatus=null, expiresAt=null for other report types. Use an injectable clock in tests. GET never writes a status or refreshes evidence. Rank reports by confirmationCount DESC, lastEvidenceAt DESC, reportId ASC for stable ties.

Cost: unique confirmer counts do not describe how many times conditions were rechecked, and a confirmation is community testimony rather than independent verification. Reverse this design if moderation/history requirements require separate evidence-event rows.

Pickup timing and disputes

Daily cutoff: a scheduled date has passed at 00:00 Africa/Lagos on the next day. No completion and past date → MISSED. No completion and today/future → SCHEDULED. Completion without conflicting resident report → DONE. Completion plus at least one MISSED_PICKUP report for the same street/date → DISPUTED. Preserve both sides; do not treat operator completion as proof that residents are wrong. A resident claim without a completion remains visible as a report; it does not make a future scheduled date MISSED.

Accept completion for scheduled dates up to today; reject future dates and unscheduled dates. Require MISSED_PICKUP reports to reference a past scheduled date. These are selected validation policies, not explicit brief wording. Permit the completion/report writes to race; derive dispute by joining both facts so the final result does not depend on write order.

Authority metrics: missedPickupCount means overdue occurrences with no completion; disputedPickupCount is separate; completedPickupCount includes completed records even if disputed. Never add disputed pickups to missed without explaining a new metric. Cost: no automatic determination of truth or dispute resolution. Reverse this decision if authorised review and evidence standards become available.

Privacy and photos

Public report responses omit reporter ID, display name, email and precise submitted coordinates. Use public street-level coordinates for map markers and return locationPrecision: 'STREET'. Store submitted location privately only when permission is given; deny precise location access to all MVP public endpoints, including authority dashboard responses. State a retention policy before using real data.

Upload optional JPEG/PNG, maximum 5 MiB, decode and re-encode server-side before storing; verify resulting bytes have no EXIF/GPS/XMP location metadata. Validate file content, dimensions and size rather than trusting filename or MIME header. Reject unsupported formats clearly. Bind photo ownership to the principal and reject another user's photo ID. Original bytes must not remain in object storage or logs. A photo may still visibly show a house number: metadata removal cannot guarantee visual anonymity. Use capture="environment" as a browser hint only; never promise gallery uploads are impossible.

7 HTTP contract and backendfrontend name map

Enter all paths below into openapi.yaml and implement them exactly as written. Prefix: /api/v1. GET operations are safe and idempotent. PUT preferences, completion and confirmation are idempotent under documented rules. POST session/photo creation is neither safe nor generally idempotent. POST report creation uses an idempotency key.

POST /auth/sessions
Roles: Unauthenticated

operationId / Java controller method: createSession / AuthController.createSession

Java service API: AuthApi.createSession

Frontend function: authApi.createSession

GET /users/me
Roles: All logged in

operationId / Java controller method: getCurrentUser / UserController.getCurrentUser

Java service API: UserApi.getCurrentUser

Frontend function: userApi.getCurrentUser

PUT /users/me/preferences
Roles: RESIDENT

operationId / Java controller method: updateMyPreferences / UserController.updateMyPreferences

Java service API: UserApi.updateMyPreferences

Frontend function: userApi.updateMyPreferences

GET /wards
Roles: All logged in

operationId / Java controller method: listWards / WardController.listWards

Java service API: WardApi.listWards

Frontend function: wardApi.listWards

GET /wards/{wardId}/streets
Roles: All logged in

operationId / Java controller method: listStreets / WardController.listStreets

Java service API: WardApi.listStreets

Frontend function: wardApi.listStreets

GET /collection-schedules
Roles: RESIDENT, AUTHORITY; OPERATOR assigned scope

operationId / Java controller method: listCollectionSchedules / CollectionController.listCollectionSchedules

Java service API: CollectionApi.listCollectionSchedules

Frontend function: collectionApi.listCollectionSchedules

GET /operators/me/streets
Roles: OPERATOR

operationId / Java controller method: listMyAssignedStreets / OperatorController.listMyAssignedStreets

Java service API: CollectionApi.listMyAssignedStreets

Frontend function: operatorApi.listMyAssignedStreets

PUT /streets/{streetId}/collections/{collectionDate}
Roles: Assigned OPERATOR

operationId / Java controller method: recordCollection / CollectionController.recordCollection

Java service API: CollectionApi.recordCollection

Frontend function: collectionApi.recordCollection

POST /photos
Roles: RESIDENT

operationId / Java controller method: uploadPhoto / PhotoController.uploadPhoto

Java service API: PhotoApi.uploadPhoto

Frontend function: photoApi.uploadPhoto

GET /photos/{photoId}
Roles: All logged in

operationId / Java controller method: getPhoto / PhotoController.getPhoto

Java service API: PhotoApi.getPhoto

Frontend function: Use returned photoUrl

POST /reports
Roles: RESIDENT

operationId / Java controller method: createReport / ReportController.createReport

Java service API: ReportApi.createReport

Frontend function: reportApi.createReport

GET /reports
Roles: All logged in

operationId / Java controller method: listReports / ReportController.listReports

Java service API: ReportApi.listReports

Frontend function: reportApi.listReports

GET /reports/{reportId}
Roles: All logged in

operationId / Java controller method: getReport / ReportController.getReport

Java service API: ReportApi.getReport

Frontend function: reportApi.getReport

PUT /reports/{reportId}/confirmations/me
Roles: RESIDENT, not reporter

operationId / Java controller method: confirmReport / ConfirmationController.confirmReport

Java service API: ConfirmationApi.confirmReport

Frontend function: reportApi.confirmReport

GET /wards/{wardId}/hazards
Roles: All logged in

operationId / Java controller method: listWardHazards / ReportController.listWardHazards

Java service API: ReportApi.listWardHazards

Frontend function: reportApi.listWardHazards

GET /wards/{wardId}/dashboard
Roles: AUTHORITY

operationId / Java controller method: getWardDashboard / AuthorityController.getWardDashboard

Java service API: AuthorityApi.getWardDashboard

Frontend function: authorityApi.getWardDashboard

Authentication: seeded accounts and password login; no self-service registration, refresh-token system or role-management UI required for MVP. Session response is { accessToken, tokenType: 'Bearer', expiresAt, user: CurrentUserView }; keep bearer token in memory and require login after reload/expiry. Document this tradeoff; do not silently invent refresh endpoints. Hash passwords and keep credentials out of logs. CurrentUserView contains userId, displayName, role, wardId, streetId; null preferences are allowed. Demo credentials belong in README, never real credentials.

8 Exact requestresponse shapes

All IDs are UUID strings. Required fields cannot be omitted or null. Optional request fields may be omitted; nullable response fields are always present with null. Do not serialise JPA entities directly.

Schema	Fields
CreateSessionRequest	required email, password strings
UpdatePreferencesRequest	required wardId, streetId
WardView	wardId, name
StreetView	streetId, wardId, name, nullable mapLocation: {latitude,longitude}
PhotoView	photoId, photoUrl, createdAt
CreateReportRequest	required reportType, streetId, note; optional photoId, location, collectionDate (conditionally required for MISSED_PICKUP)
ReportView	reportId, reportType, wardId, streetId, streetName, note, photoUrl, createdAt, collectionDate, confirmationCount, confirmedByMe, lastEvidenceAt, floodStatus, expiresAt, nullable street mapLocation, locationPrecision: 'STREET'
CollectionScheduleView	scheduleId, wardId, streetId, streetName, collectionDate, collectionStatus, nullable completedAt, missedReportCount; operator identity excluded from resident/operator public views
CollectionView	collectionId, streetId, collectionDate, completedAt, collectionStatus
WardHazardsView	wardId, generatedAt, nullable newestReportAt, items: ReportView[] (CURRENT floods and blocked drains)
WardDashboardView	wardId, fromDate, toDate, generatedAt, missedPickupCount, disputedPickupCount, blockedDrainCount, currentFloodCount, nullable newestReportAt, operatorPerformance: OperatorPerformanceView[]
OperatorPerformanceView	operatorId, fictional operatorName, scheduledPickupCount, completedPickupCount, missedPickupCount, disputedPickupCount
Page<T>	items: T[], zero-based page, size, totalElements, totalPages

Create report example:

```
{
  "reportType": "BLOCKED_DRAIN",
  "streetId": "ae8cf275-e382-4605-91db-8ed479916f39",
  "note": "Refuse is blocking the drain near the junction.",
  "photoId": null,
  "location": null
}
```

For optional request fields, explicit null is accepted only for photoId and location; collectionDate is omitted for non-missed reports. Required note: trim, 1–1000 characters. Server sets timestamps. Coordinates must be a complete pair, latitude −90..90 and longitude −180..180; street must exist. Precise coordinates are never echoed publicly.

Filters and ordering:

Endpoint	Query parameters and defaults
/reports	required wardId; optional streetId, reportType, floodStatus; sort=CONFIRMATIONS default or NEWEST; page=0, size=20 (1–100)
/collection-schedules	required wardId, fromDate, toDate; optional streetId; date range inclusive, max 31 days; pageable response
/operators/me/streets	No caller-supplied operator ID; returns StreetView[] for principal's assigned wards
/wards/{wardId}/hazards	No performance data; all relevant ward hazards in MVP one-ward dataset; do not silently truncate; later paginate if scope expands
/wards/{wardId}/dashboard	required inclusive fromDate, toDate, max 31 days; pickup metrics use that range; hazard counts are current snapshot as of generatedAt

floodStatus without reportType=FLOODED_ROAD returns 400. Cross-ward street filters return 400. NEWEST sorts createdAt DESC, reportId ASC. Blocked-drain count means all recorded blocked-drain reports because resolution is not in MVP; label “Blocked-drain reports”, not “Verified unresolved drains”. Current flood count counts reports, not distinct roads; use that wording. Operator performance never leaks through general schedules, report projections or logs exposed to clients.

9 Status codes errors and repeat submission

Operation	Success	Expected failures
Login	200 session	400 invalid shape; 401 bad credentials
Reads	200 including empty collections	400 bad filters; 401 unauthenticated; 403 forbidden; 404 missing resource
Preferences	200 current user	400 inconsistent ward/street; 401/403
Photo upload (multipart/form-data, field file)	201 PhotoView	400 invalid image; 413 too large; 415 unsupported; 401/403
Create report	201 ReportView + Location header; replay returns same 201/body	400 validation/missing key; 403 role/photo ownership; 404 street; 409 key mismatch
Record collection (empty body)	201 first insert; 200 existing record, no timestamp overwrite	400 future/unscheduled date; 403 wrong role/ward; 404 street
Confirm report (empty body)	200 authoritative ReportView for first/repeat/reconfirmation	401/403; 404 report; 409 self-confirmation

Uniform JSON error, content type application/problem+json:

```
{
  "type": "urn:ekowatch:problem:validation",
  "title": "Validation failed",
  "status": 400,
  "code": "VALIDATION_FAILED",
  "detail": "Check the highlighted fields.",
  "instance": "/api/v1/reports",
  "requestId": "d6c37192-5428-442d-a067-86c7336a842d",
  "fieldErrors": [{ "field": "note", "message": "Note is required." }]
}
```

Stable codes: VALIDATION_FAILED, UNAUTHENTICATED, FORBIDDEN, RESOURCE_NOT_FOUND, WARD_ASSIGNMENT_REQUIRED, SELF_CONFIRMATION_NOT_ALLOWED, IDEMPOTENCY_KEY_REQUIRED, IDEMPOTENCY_KEY_REUSED, PHOTO_TOO_LARGE, UNSUPPORTED_PHOTO_TYPE, INTERNAL_ERROR. No SQL messages, stack traces, tokens, names or private coordinates in error bodies.

For report creation, require Idempotency-Key: <UUID>. Scope it by authenticated user and operation; store canonical request hash and original response in idempotency_records with unique (user_id,operation,idempotency_key). Create report and completed idempotency response in one transaction. Concurrent same-key requests wait/replay; changed payload gets 409. Retention: 24 hours, stated in OpenAPI/README. Client generates one key per user intent, keeps it with pending payload until resolved, retries with the same key, and creates a new key for a genuinely new report. Lost response means “Outcome unknown—check or retry”, not a definite failure. Photo upload occurs first; retry report with the same photo ID. Orphan-photo cleanup is optional but storage limits must be monitored.

Every call sends X-Request-ID; backend validates or replaces it and echoes it. Logs contain operation, outcome, duration and requestId, without report notes or private location.

10 Backend structure and responsibilities

Base package: com.tesa.ekowatch. Modules: user, ward, collection, report, confirmation, photo, authority, shared. Each uses api, domain, repository, service, web where needed. Put interfaces/views in api, entities in domain, controllers in web. Avoid two competing public interfaces named ReportApi and ReportServiceApi.

Controllers authenticate, bind and validate input, call services, and map results/statuses. Services enforce role, ward assignment, business rules and transaction boundaries even when invoked outside controllers. Repositories provide atomic writes/locking; database constraints are the final guard. Frontend validation improves usability but never substitutes for backend rules.

Derive confirmation counts directly from report_confirmations rows. Do not load a count, increment it in Java and overwrite it. For concurrent same-street completion use an insert conflict path and fetch existing record; avoid catching a constraint failure and continuing in an already-aborted transaction. For report/confirmation writes use a consistent locking order. No scheduler is required for missed pickups or flood expiry.

11 Frontend routes modules and states

Browser route	Component	Data function / main action
/login	LoginPage	authApi.createSession
/setup	WardSetupPage	listWards, listStreets, updateMyPreferences
/collections	CollectionSchedulePage	listCollectionSchedules
/reports	ReportListPage	listReports with combined filters
/reports/new	CreateReportPage	optional uploadPhoto, then createReport
/reports/:reportId	ReportDetailPage	getReport, confirmReport
/hazards	HazardsPage	listWardHazards; map/list switch

Browser route	Component	Data function / main action
/operator/collections	OperatorCollectionsPage	listMyAssignedStreets, schedules, recordCollection
/authority/dashboard	AuthorityDashboardPage	getWardDashboard

Generate TypeScript models from OpenAPI into src/types/ekowatch.ts. Use src/lib/api/http.ts, domain API modules and page containers. Components receive props/callbacks; they do not invent response fields. Use one API base URL setting, VITE_API_BASE_URL, including /api/v1 once. Browser routes are not HTTP endpoint names. Use TanStack Query for server data and React component state for local form state. Use React Router for navigation and Tailwind CSS for styling.

All MVP journey screens need loading, empty, error, success and confirmation states. Confirmation means a clear acknowledgement/review of the intended action; do not add unnecessary modal clicks to every read. Forms retain entered values after a failed save. Disable duplicate in-flight submit, but rely on server repeat protection too. Show actionable 400 fields, 401 login, 403 permissions, 404 missing resource and 409 conflict messages. An offline map must not hide the list.

Optimistic action: confirmation. Mark it pending and increment only if confirmedByMe=false; reconfirmation refreshes evidence without incrementing people count. On explicit rejection restore prior state and explain. On a lost connection retain an “Unconfirmed save” state and refetch/retry before asserting success or rolling back as fact. Replace cache with server response and invalidate report list, hazards and authority dashboard. Avoid putting private identity in shared report caches.

Use server generatedAt/timestamps for age labels, not browser-created evidence times. Show “Community report · last confirmed … ago”; distinguish expired from current, and show newest report age on map and any later alerts. “No current flood reports” does not mean “Roads are safe”. Render statuses with text as well as colour. Provide semantic headings, labelled inputs, visible focus, keyboard map/list controls and announced success/error updates. Verify phone layout and throttled/failed-network behaviour.

12 Seeddata contract

Required dataset: one ward, at least eight streets, thirty days of schedule coverage, operators/residents and reports covering reachable states, including an expired flood and disputed pickup. Real place names may be used; operator names, phone numbers and activity must be fictional and labelled. Do not claim agency affiliation.

Seed fixtures:

- One authority, two operators assigned to the ward, and at least four residents so one reporter and two independent confirmers are available. Add an unassigned operator for refusal checks.
- At least eight streets, including a long name and a street with no reports.
- Dated collection occurrences covering thirty consecutive days with completed, past missed, today scheduled, future scheduled and disputed examples.
- All four report types; photo and no-photo cases; location permitted and denied; no confirmations and several confirmations; current and expired flood evidence.
- A completed collection and a MISSED_PICKUP report for its exact street/date, yielding DISPUTED.

Anchor relative seed times to an explicit seed clock and record it. Rerunning seeds must not keep refreshing all evidence; provide a deliberate demo reset command. Because current reports naturally expire, prepare a reproducible fresh-demo seed/reset before presentation and keep an intentionally expired fixture. Never manually edit database rows during a demonstration.

13 Build order and acceptance gates

This is dependency order, not a claim of current completion. First audit the existing code and mark each slice existing/partial/missing; preserve compatible work.

0. Reconcile contracts
Data/rules (Adedayo): Map existing columns to this vocabulary

API/access (Esther): Reconcile all paths/DTOs into OpenAPI

Frontend (Muizz): Inventory existing calls/types and mismatches

Gate before next slice: One agreed contract; discrepancies logged; no independent renames

1. Identity and setup
Data/rules (Adedayo): User/ward/street seeds and constraints

API/access (Esther): Login, principal, role guards, preferences

Frontend (Muizz): Login and ward/street setup

Gate before next slice: Three demo roles work; direct wrong-role test refuses

2. Report journey
Data/rules (Adedayo): Reports, confirmations, unique constraint and race test

API/access (Esther): Report/photo/confirmation endpoints

Frontend (Muizz): Create → details → second resident confirms

Gate before next slice: Authority later sees same persisted report; privacy checked

3. Collection journey
Data/rules (Adedayo): Schedule/collection uniqueness and assignment rules

API/access (Esther): Schedule/assignment/completion endpoints

Frontend (Muizz): Schedule and operator completion screens

Gate before next slice: Past unrecorded pickup missed; repeat completion one row

4. Freshness and dispute
Data/rules (Adedayo): Clock-based derivations and dispute query

API/access (Esther): Hazard/dashboard projections

Frontend (Muizz): Current/expired hazards, list/map, authority metrics

Gate before next slice: Expired evidence and disputed pickup shown correctly

5. Delivery evidence
Data/rules (Adedayo): Reproducible seeds, PostgreSQL verification

API/access (Esther): Gated Actions, migrations, deployment and API checks

Frontend (Muizz): Phone/keyboard/states/network verification

Gate before next slice: Clean-browser deployed acceptance; README and two ADRs

local verification commands (confirm wrappers and scripts exist before publishing README): backend Windows .\mvnw.cmd verify; frontend npm ci, npm run lint, npm run typecheck, npm run build. PostgreSQL integration checks must use real Postgres; H2 alone cannot prove production unique/locking behaviour. GitHub Actions blocks deployment on failed tests/type checking; execute migrations before serving code that needs the new schema. Contract validation/API smoke checks are recommended extra protection against the FoodBridge naming problem, even though a generated mock and full contract-test pipeline are not mandatory for a side project.

14 Verification matrix

Check	Expected result / proof
Blocked-drain journey	Resident A creates, B confirms, authority sees the report and derived count on deployed URLs

Check	Expected result / proof
Two different residents concurrently confirm	Two rows, count +2, no lost update
Same resident concurrently confirms twice	One row, count +1, valid repeat response
Protection removal experiment	In disposable test schema only, remove unique constraint and show duplicate-confirmation test fails; production untouched
Self confirmation	409 with stable code and no row created
Flood boundary	Just before expiry CURRENT; exactly at/after expiry EXPIRED; GET does not write
Reconfirmation	Existing confirmer renews expired flood; count unchanged; immediate retry does not renew again
One-hour outage	Subsequent read derives overdue pickups and flood expiry without jobs
Collection race	Same street/day yields one completion and stable original timestamp
Wrong operator/role	Direct service/API call for unassigned ward refused; authority data forbidden to residents/operators
Dispute race	Completion and matching missed report in either order yield DISPUTED, with both records intact
Privacy	Every report/list/dashboard/error response omits reporter identity and precise coordinates; saved photo has no location metadata
Repeat report creation	Same key/payload yields one report; changed payload same key yields 409; lost-response retry recovers result
Dashboard consistency	Counts match SQL fixtures; expired floods excluded; disputed separated; names fictional
Contract drift	Paths, JSON casing/enums, required/null fields, statuses and examples match OpenAPI and typed client
Frontend failure states	Empty first use, slow read, validation refusal, permission denial and unknown save outcome are usable
Delivery	Clean clone and seed run work; Actions deploys both tiers; clean browser completes journey

The side-project standard generally asks for one concurrent-rule test. EkoWatch's own definition of done specifically asks for the confirmation test to fail when the unique constraint is removed, so retain that stronger product-specific evidence. The side-project authorisation test must also fail when its check is removed. Keep those controlled experiments out of deployable code.

15 Delivery checklist and limits

- ☐ Two live addresses: frontend and HTTP service, both deployed through GitHub Actions, tests and type checks blocking deployment.
- ☐ OpenAPI covers every MVP path, DTO, enum, error and repeat-submission rule; frontend uses that contract.
- ☐ Required journeys and every EkoWatch business rule above verified; PostgreSQL concurrency evidence retained.
- ☐ One ward, eight streets minimum and thirty days of schedule coverage; demo credentials and fictional-data labels supplied.
- ☐ Five states on the MVP journey; phone layout, keyboard operation and visible focus verified.

- ☐ README includes setup, seed/reset, live addresses, demo logins, chosen freshness window/timezone, disputes, photo limitations and known omissions.
- ☐ Two ADRs contain context, decision, rejected alternative, accepted cost and observation that would reverse the decision.
- ☐ Entry in team AI log; engineers can explain code and decisions they submit.
- ☐ Free hosting terms checked directly with providers before selection, including sleep, expiry, quotas and number of projects. This plan does not assert that any named provider is currently free.
- ☐ No secrets committed; no real resident/operator data in demo seeds.

Not required for this side project: full load/saturation test, screen-reader report, Core Web Vitals targets, five ADRs or walkthrough video. Those are main-product requirements and should not displace EkoWatch's required scope. Additional contract checks, structured logging and real-Postgres coverage in this plan are engineering recommendations tailored to integration risks.

16 Naming enforcement

Implement the tables, Java APIs, controllers, operation IDs, JSON fields, frontend functions and routes exactly as listed. Do not introduce synonyms or aliases. Database columns use snake_case; JSON, Java members and TypeScript properties use lowerCamelCase for the same concept. This intentional case conversion is the only naming variation.

Before implementation, compare the current code with the baseline and list required migrations in docs/decisions.md. Change the contract, backend and frontend together in a reviewed change. This document does not claim that the existing repositories already follow these names.